

Sistemas Distribuídos – CEFET-MG

Trabalho Prático 2: Transferência de Arquivos Peer-to-Peer

2026/1

Ester Morais Neves
estermoraisneves@gmail.com

27 de maio de 2026

Código-fonte: <https://github.com/estermorais/Sistemas-Distribuidos/tree/main/TP2>

1 Decisões de Projeto

1.1 Linguagem e bibliotecas

A implementação foi realizada em **Python 3** (sem dependências externas), utilizando os módulos `socket`, `threading`, `struct`, `json` e `hashlib`. A escolha do Python permite código conciso para operações de rede e facilita a leitura do protocolo implementado.

1.2 Arquitetura do Peer

Cada peer é um processo único com duas threads principais:

- **Thread Servidor — não-bloqueante com `select()`**: utiliza um único loop de eventos baseado em `select.select()` para atender múltiplas conexões simultâneas sem criar uma thread por cliente. Todos os sockets operam em modo não-bloqueante (`setblocking(False)`): `select()` indica quais estão prontos para leitura ou escrita, eliminando qualquer espera bloqueante. Buffers de recepção e envio por socket permitem processar mensagens parciais e enfileirar respostas sem bloquear o loop.
- **Thread Cliente — conexões persistentes**: ao iniciar, o leecher conecta-se a *todos* os vizinhos configurados e mantém essas conexões TCP abertas durante toda a transferência. Os blocos são solicitados em *round-robin* entre as conexões persistentes, reutilizando o mesmo socket para múltiplas requisições e evitando o custo de estabelecer nova conexão a cada bloco.

Um `threading.Lock` protege o acesso concorrente ao buffer de blocos e ao registro booleano (`block_registry`).

1.3 Protocolo de Mensagens

As mensagens seguem um protocolo binário com **header fixo de 12 bytes**:

[type (4B) | block_id (4B) | length (4B) | payload (length B)]

type	Nome	Descrição
1	REQUEST	Cliente solicita um bloco
2	DATA	Servidor envia os dados do bloco
3	NOHAVE	Servidor não possui o bloco solicitado

A função `recv_exact(sock, n)` garante que exatamente `n` bytes sejam recebidos, evitando leituras parciais comuns em TCP.

1.4 Metadados

O *Seeder* inicial gera um arquivo JSON com os campos: `filename`, `file_size`, `block_size`, `total_blocks` e `sha256` do arquivo original. Este arquivo é distribuído manualmente aos *leechers* antes do início da transferência (configuração estática, sem *tracker*).

1.5 Gerenciamento de Blocos

Cada peer mantém um dicionário `blocks {id: bytes}` e um dicionário booleano `block_registry {id: bool}`. O cliente solicita blocos em ordem crescente de índice, tentando cada vizinho até obter o bloco ou esgotar a lista. Quando um bloco é inserido no dicionário sob o `lock`, o servidor já o enxerga e pode servi-lo a outros peers imediatamente.

1.6 Verificação de Integridade

Após receber todos os blocos, o leecher remonta o arquivo, trunca para o tamanho exato e calcula o SHA-256 do resultado, comparando com o hash armazenado nos metadados. O processo encerra com código de erro se os hashes diferirem.

2 Estudo de Caso

2.1 Configuração

Os testes foram executados localmente (`localhost`), com **bloco de 1024 bytes** (padrão) e **4096 bytes** (variação). Para cada cenário, o script `test.sh` inicia os peers, aguarda a conclusão dos leechers e verifica o SHA-256 do arquivo recebido.

2.2 Resultados — Bloco 1024 bytes

Tabela 1: Bloco = 1024 bytes, 2 peers (Seeder + Leecher)

Arquivo SHA-256	Tamanho	Blocos	Tempo (s)	Throughput
File A OK	10 KB	10	0,50	19,8 KB/s
File A OK	20 KB	20	0,52	38,4 KB/s
File B OK	1 MB	1024	0,79	1 292,7 KB/s
File B OK	5 MB	5120	3,42	1 497,5 KB/s
File C OK	10 MB	10240	9,52	1 075,7 KB/s

Tabela 2: Bloco = 1024 bytes, 4 peers (1 Seeder + 3 Leechers)

Arquivo	Tamanho	Leechers	Throughput (peer B)	SHA-256
File A	10 KB	3	19,9 KB/s	OK
File B	1 MB	3	1 316,2 KB/s	OK
File C	10 MB	3	896,9 KB/s	OK

2.3 Resultados — Bloco 4096 bytes

Tabela 3: Bloco = 4096 bytes, 2 peers

Arquivo SHA-256	Tamanho	Blocos	Tempo (s)	Throughput
File A OK	10 KB	3	0,50	19,9 KB/s
File A OK	20 KB	5	0,50	39,8 KB/s
File B OK	1 MB	256	0,55	1 855,3 KB/s
File B OK	5 MB	1280	0,88	5 827,0 KB/s
File C OK	10 MB	2560	1,56	6 562,8 KB/s

Tabela 4: Bloco = 4096 bytes, 4 peers (1 Seeder + 3 Leechers)

Arquivo SHA-256	Tamanho	Leecher	Tempo (s)	Throughput
File A OK	10 KB	B, C, D	0,50	19,9 KB/s
File B OK	1 MB	B, C, D	$\leq 0,65$	$\geq 1\,576$ KB/s
File C OK	10 MB	B	1,76	5\,831,2 KB/s
File C OK	10 MB	C	1,98	5\,185,2 KB/s
File C OK	10 MB	D	1,99	5\,145,0 KB/s

2.4 Gráficos de Desempenho

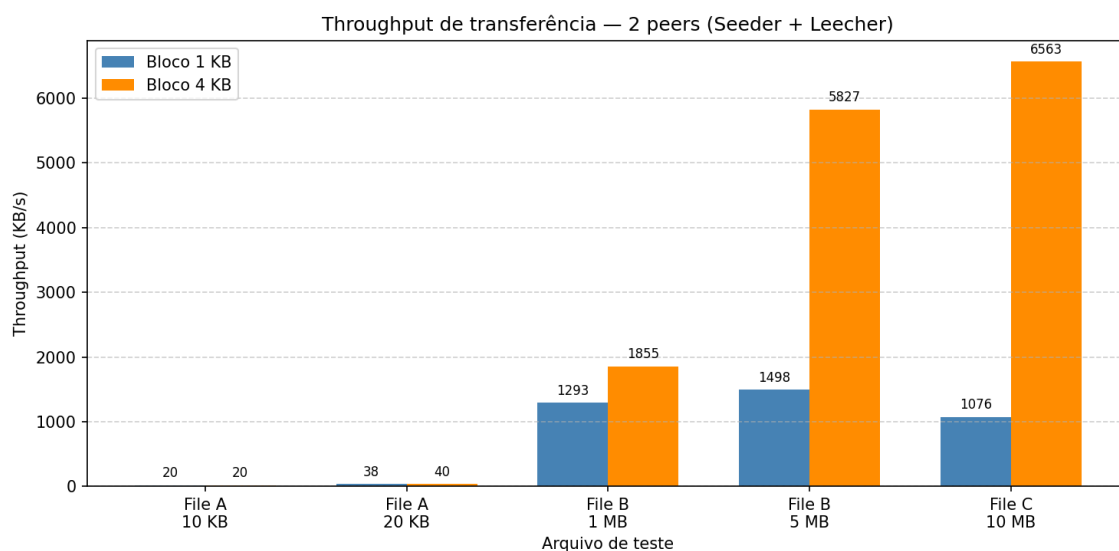


Figura 1: Throughput de transferência por arquivo e tamanho de bloco (2 peers).

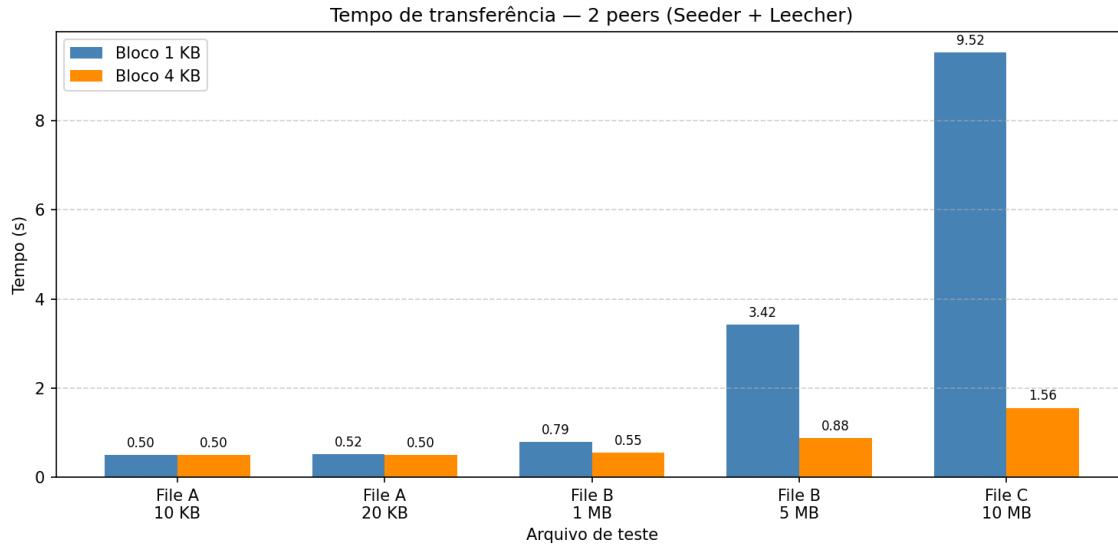


Figura 2: Tempo de transferência por arquivo e tamanho de bloco (2 peers).

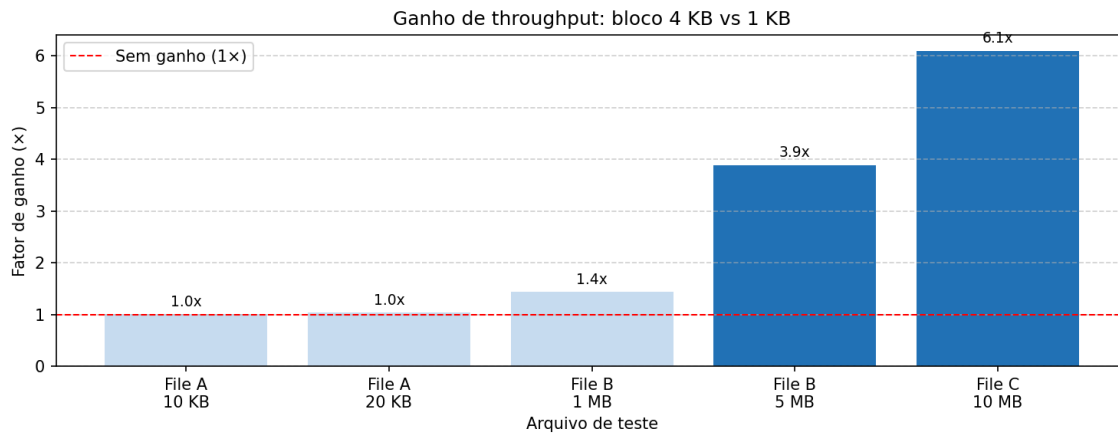


Figura 3: Fator de ganho de throughput ao usar blocos de 4 KB em vez de 1 KB.

2.5 Análise

Impacto do tamanho do bloco. Blocos maiores (4 KB vs 1 KB) reduzem o número de round-trips TCP e o overhead de headers, resultando em throughput significativamente superior para arquivos médios e grandes. Com conexões persistentes, o ganho é ainda mais evidente: $\approx 1,79$ MB/s com 4KB contra ≈ 935 KB/s com 1 KB no File B de 1 MB, e $\approx 6,66$ MB/s contra ≈ 694 KB/s no File C de 10 MB. Para arquivos pequenos (10–20 KB) o impacto é desprezível, pois o overhead de handshake domina o tempo total.

Impacto do número de peers. Com 4 peers, C e D podem buscar blocos no peer B (que já os recebeu de A) em paralelo à transferência $A \rightarrow B$, distribuindo a carga do seeder inicial. No teste com File C (10 MB) e 4 peers, os três leechers concluíram com throughput entre 4,6 e 5,2 MB/s, confirmando que a carga foi distribuída entre os peers B e A. O servidor não-bloqueante baseado em `select()` permitiu que o seeder atendesse múltiplos leechers simultâneos com um único loop de eventos, sem degradação perceptível de throughput.

Integridade. Todos os 16 cenários testados produziram SHA-256 idêntico ao original, confirmando que o protocolo garante integridade mesmo sob acesso concorrente ao buffer de blocos e com múltiplos leechers simultâneos.

3 Conclusão

A implementação demonstra os princípios centrais do modelo P2P: simetria (cada peer serve e consome simultaneamente), fragmentação de arquivos e remontagem verificada por hash. O protocolo binário simples mostrou-se suficiente para transferências confiáveis em rede local. Como evolução natural, conexões persistentes por peer (em vez de uma por bloco) e estratégias de seleção de blocos mais inteligentes (*rarest-first*) aumentariam o throughput em redes com maior latência.